compile-sast

Vulnerabilities as build failures, not CI failures.

Whitney Zilton

BSides Houston · September 17, 2026

WHY

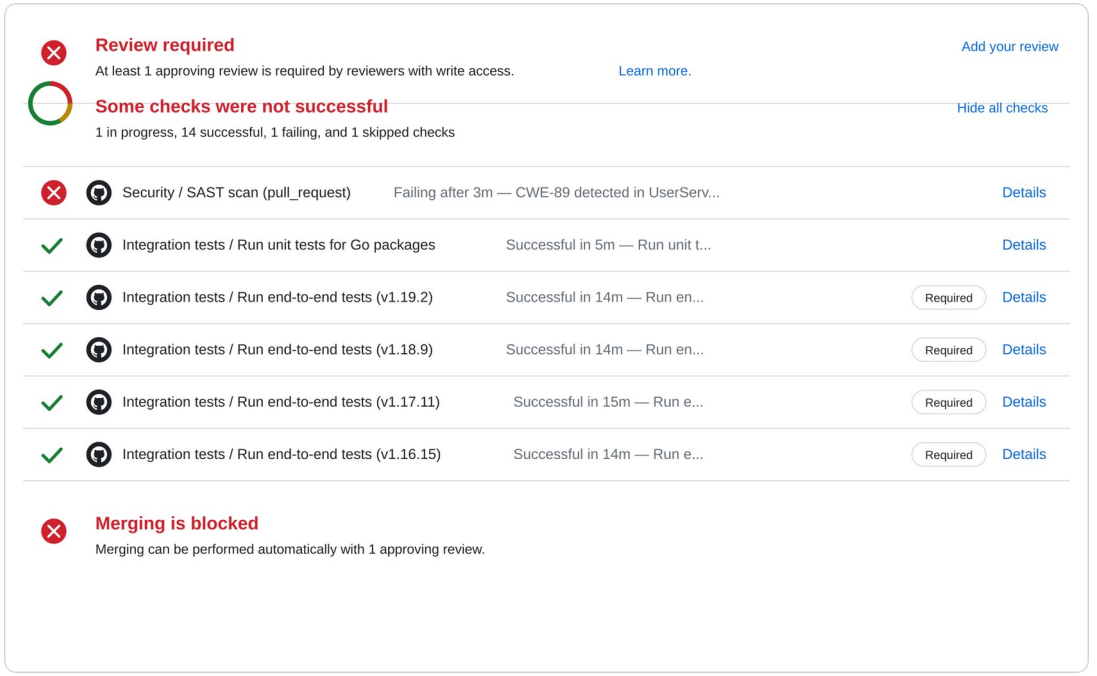
Motivation for building compile-sast

I finished the Jira ticket in my head.

Code compiles. Tests all pass. Configs are in order. I'm done, right? Right?!

Time to push up my branch… only to see the SAST scan in my ci/cd pipeline failed.

How can I find security vulnerabilities earlier in the SDLC?



Background

Backend software engineer with over 5 years of experience.

I've worked in lots of teams on different languages: Go, Java, Kotlin, Python. Lots of Kafka-based microservices.

I've led threat modeling exercises, SAST/DAST/SCA integration, pentest coordination, and CVE remediation on teams.

Software engineer who cares about security 😊

RELEVANCE

Why should I care?

LLMs suggest what they've seen.

Training data is full of MD5 hashes, string-concat SQL, hardcoded secrets in tutorial code. AI assistants happily reproduce those patterns.

"Looks reasonable" isn't "is secure."

AI suggestions get accepted at higher rates than the same code would be written from scratch. Plausibility bypasses skepticism.

Volume goes up. Review capacity doesn't.

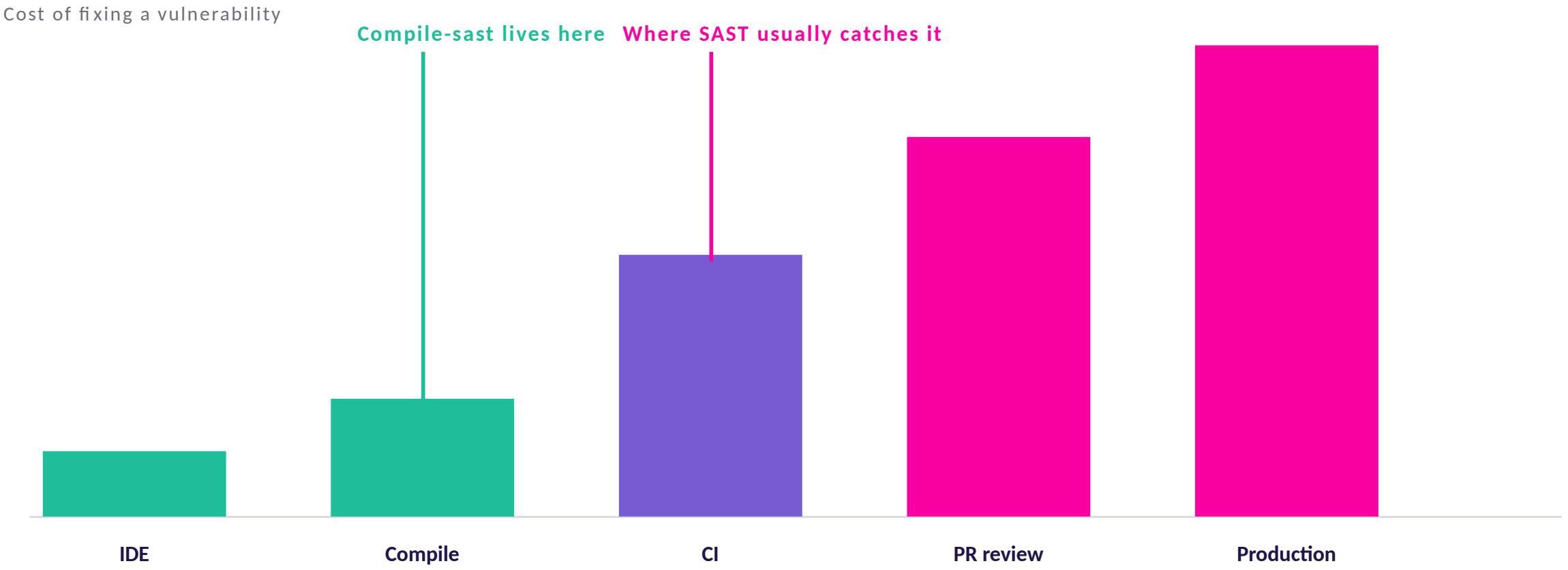
More code lands per week. The same reviewers still have to catch the vulnerable patterns among all the safe ones.

A compile rule doesn't care who typed it.

The same guardrail applies to your code, your teammate's code, and whatever the AI wrote last Tuesday.

FEEDBACK

The later the signal, the more it costs



THESIS

If it can be a compile error, it should be a compile error.

The compiler is the earliest, cheapest, most trusted feedback surface a developer touches.

PROJECT INFO

What is compile-sast?

LANGUAGE

Kotlin

K2 / FIR compiler plugin

MAPPINGS

10 OWASP/CWE-mapped vulnerability classes

Enforced at build time

LICENSE

MIT

permissive, portfolio-friendly

DISTRIBUTION

Maven Central

+ Gradle Plugin Portal

SCOPE

What is compile-sast not?

Not a full SAST

No taint tracking, no cross-procedural dataflow, no whole-program analysis. Semgrep, Snyk Code, and CodeQL still catch things this can't.

Not a dependency scanner

compile-sast checks your source. Third-party libraries and transitive dependencies are still your SCA tool's job.

Not language-agnostic

Kotlin only, tied to the K2/FIR checker API. The idea generalizes; the plugin doesn't.

Not exhaustive

Each rule targets specific syntactic patterns, not every possible expression of its CWE. False negatives are real: that's the trade-off for being safe to make blocking.

Complementary, not competitive. Keep this in your build; keep your SAST and SCA in CI.

Why Kotlin?

Kotlin

MECHANISM

K2 / FIR checker API

AVAILABILITY

New in Kotlin 2.0

What compile-sast is built on. First-class compiler plugin surface.

C#

MECHANISM

Roslyn analyzers

AVAILABILITY

Standard since ‘15

.NET has done this for a decade. The template is proven.

Go

MECHANISM

Out-of-compiler

AVAILABILITY

staticcheck, gosec, golangci-lint

Different model: analysis runs alongside the build, not inside it.

C# had this. Go’s ecosystem chose a different model. Kotlin K2 finally opened the door, *and I’ve written in Kotlin for two years*.

How it plugs in:



Same phase as the type checker.

ERROR findings fail the build; WARNING findings are reported. No separate scanner to run, no CI/CD step to wait on. The same feedback surface Kotlin developers already trust.

COVERAGE

Ten classes, enforced at build time

CWE-798	Hardcoded credentials
CWE-89	SQL injection
CWE-78	OS command injection
CWE-22	Path traversal
CWE-327	Weak cryptographic algorithm
CWE-326	Weak TLS/SSL protocol
CWE-611	XML external entities
CWE-502	Insecure deserialization
CWE-532	Logging sensitive data
CWE-918	Server-side request forgery

Hardcoded credentials

VulnerableSamples.kt

```kotlin
package com.compile.sast.sample

import java.security.MessageDigest

/**
 * Sample class demonstrating
 * vulnerable patterns caught
 * by compile-sast rules at
 * build time. Each property
 * or function fails the build
 * with a specific diagnostic.
 */
object VulnerableSamples {
    val apiKey = "sk-live-abc123def456"

    // SQL injection via
    // string interpolation
    fun query(id: String) =
        "SELECT * FROM u " +
        "WHERE id = '$id'"

    // Weak cryptography
    val digest = MessageDigest
        .getInstance("MD5")
}
```

```
adinah@Mac compile-sast % ./gradlew :sample-app:compileKotlin

> Task :sample-app:compileKotlin FAILED
e: file:///Users/adinah/Projects/compile-sast/sample-app/src/main/kotlin/com/compile/sast/sample/VulnerableSam
ples.kt:14:5 SAST001 [CWE-798] Hardcoded secret: property 'apiKey' is initialized with a hardcoded secret-shap
ed literal

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':sample-app:compileKotlin'.
> A failure occurred while executing org.jetbrains.kotlin.compilerRunner.GradleCompilerRunnerWithWorkers$Gradl
eKotlinCompilerWorkAction
    > Compilation error. See log for more details

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

BUILD FAILED in 1s
1 actionable task: 1 executed
```

EXAMPLE · CWE-89

SQL injection via string concatenation

VulnerableSamples.kt

```kotlin
class SqlInjectionExample {
    // Concatenation: the argument is a `plus` call, not a string literal.
    fun vulnerableQuery(connection: Connection, userId: String): String {
        val statement = connection.createStatement()
        val resultSet = statement.executeQuery("SELECT * FROM users WHERE id = '" + userId + "'")
        return resultSet.toString()
    }

    // String template: also not a compile-time constant, even though the
    // interpolation is spread across two concatenated lines.
    fun findByEmail(connection: Connection, email: String): String {
        val sql = "SELECT * FROM users " +
                "WHERE email = '$email'"
        val statement = connection.createStatement()
        return statement.executeQuery(sql).toString()
    }
}
```

```
adinah@Mac compile-sast % ./gradlew :sample-app:compileKotlin

> Task :sample-app:compileKotlin FAILED
e: warnings found and -Werror specified
w: file:///Users/adinah/Projects/compile-sast/sample-app/src/main/kotlin/com/compile/sast/sample/VulnerableSam
ples.kt:47:25 SAST003 [CWE-89] Potential SQL injection: query argument to 'executeQuery' is not a compile-time
 constant string
w: file:///Users/adinah/Projects/compile-sast/sample-app/src/main/kotlin/com/compile/sast/sample/VulnerableSam
ples.kt:57:16 SAST003 [CWE-89] Potential SQL injection: query argument to 'executeQuery' is not a compile-time
 constant string

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':sample-app:compileKotlin'.
> A failure occurred while executing org.jetbrains.kotlin.compilerRunner.GradleCompilerRunnerWithWorkers$Gradl
eKotlinCompilerWorkAction
    > Compilation error. See log for more details

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

BUILD FAILED in 837ms
1 actionable task: 1 executed
```

Weak cryptographic algorithm

VulnerableSamples.kt

```kotlin
package com.compile.sast.sample

import java.security.MessageDigest

/**
 * Sample class demonstrating
 * vulnerable patterns caught
 * by compile-sast rules at
 * compile time. Each function
 * fails the build with its
 * own SAST diagnostic.
 */
object VulnerableSamples {

    // Hardcoded credential
    val apiKey = "sk-abc123"

    // SQL injection via
    // string interpolation
    fun query(id: String) =
        "SELECT * FROM u " +
        "WHERE id = '$id'"

    // Weak cryptography
    val digest = MessageDigest.getInstance("MD5")
}
```

```
adinah@Mac compile-sast % ./gradlew :sample-app:compileKotlin
e: file:///Users/adinah/Projects/compile-sast/sample-app/src/main/kotlin/com/compile/sast/sample/VulnerableSam
ples.kt:25:22 SAST002 [CWE-327] Weak cryptographic algorithm: 'MD5' uses weak algorithm/mode 'MD5'

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':sample-app:compileKotlin'.
> A failure occurred while executing org.jetbrains.kotlin.compilerRunner.GradleCompilerRunnerWithWorkers$Gradl
eKotlinCompilerWorkAction
    > Compilation error. See log for more details

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

BUILD FAILED in 911ms
1 actionable task: 1 executed
```

LIVE

Demo

What you're about to see

A fresh Kotlin project with a clean build.
Introducing vulnerabilities and watching the build fail.
Fixing them and watching the build come back green.

SCREENSHOT

Just in case the live demo doesn’t work :’)

```
adinah@Mac compile-sast % ./gradlew :sample-app:compileKotlin
 constant string

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':sample-app:compileKotlin'.
> A failure occurred while executing org.jetbrains.kotlin.compilerRunner.GradleCompilerRunnerWithWorkers$Gradl
eKotlinCompilerWorkAction
   > Compilation error. See log for more details

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

BUILD FAILED in 1s
1 actionable task: 1 executed
```

Running ./gradlew :sample-app:compileKotlin in terminal

Try it in your project

1

Add the plugin

One line in build.gradle.kts

2

Rebuild

Existing rules run on your source

3

Read the diagnostics

ERROR fails the build, WARNING reports

build.gradle.kts

```
apply the plugin
plugins {
    kotlin("jvm") version "2.1.10"
    id("io.github.adinahhh.compile-sast") version "0.1.0"
}

resolve the compiler plugin from Maven Central
repositories {
    mavenCentral()
}
```

FUTURE ROADMAP

What's next

More rule classes

The next wave targets adjacent gaps: insecure JWT usage, missing session-cookie flags, and framework-specific access-control patterns.

Taint tracking added

Moves the project from a PoC to a legitimate open source SAST scanning tool.

Beyond Kotlin

The K2/FIR approach is Kotlin-specific by design, but the same shape works for any typed language with a plugin API.

SCA compiler plugin

I’m curious about adding more compiler plugins. An SCA scan for third party vulnerabilities (both direct and transitive) would be really cool to explore building.

Let's keep talking.

- Which rules would you want to be blocking vs. warning?
- What's the boundary between "compiler's job" and "linter's job"?
- How would you sell build-time enforcement to a skeptical team?
- What are some of your thoughts on Shift Left philosophy?

BYE

Connect with me ☺

Github:

https://github.com/adinahhh/compile-sast

Portfolio:

https://ziltonportfolio.io/